



Performance Management System

The logic book — how every screen works, what it reads and writes, and the rules that decide whether it lets you.

Prepared for Mindgate

Version of 06 October 2026 · build 4e9d532 + Short-Term from the matrix

Covers: the appraisal cycle, role sections, Dashboard, KRAs, growth, connects, reviews, calibration, timesheet ratings, competencies, Super 50, Rewards & Recognition, engagement and the First-Week Journey.

Contents

Part 1 — The engine

- 1.1 The cycle, and its nine phases
 - What each phase opens
- 1.2 Who may open a screen
- 1.3 Whose rows you see
- 1.4 Four house rules visible on every screen

Part 2 — The five role sections

- 2.1 Self — 13 pages plus Dashboard, everyone
- 2.2 Manager — 9 pages, pms_team_eval
- 2.3 HOD — 2 pages, pms_hod
- 2.4 HRBP — 20 pages, pms_hrbp
- 2.5 HR — 22 pages
- 2.6 Dashboard

Part 3 — The machinery

- 3.1 The KRA sheet status machine
- 3.2 Growth planning opens per person
- 3.3 Ratings: numbers in the database, letters on screen
- 3.4 Calibration and the kitty
- 3.5 Increment simulation models pay, it does not set it
- 3.6 Who is in this cycle
- 3.7 Timesheet → KRA
 - Timesheet rating per KRA (6 Oct)
- 3.8 Competencies
- 3.9 Super 50
- 3.10 Rewards & Recognition
- 3.11 Engagement and anonymity
- 3.12 First-Week Journey — onboarding tracker (New Hire Insights, HR and HRBP; HR Ops)
- 3.13 Reminders

Part 4 — Changing behaviour without code

Part 5 — What it deliberately does not do

How the product actually works, section by section: what each screen reads, what it writes, and the rules that decide whether it lets you.

It is written for somebody who has to answer "why did it do that?" — a reviewer, a new developer, or HR working out whether a behaviour is a bug or a decision. Every rule below names the file that enforces it, so the book can be checked against the code rather than believed.

Read Part 1 first. Four mechanisms govern every screen in the product. Once those are clear, most of Part 2 is predictable.

Live at pms.agentichumans.in: build [4e9d532](#) (7 October 2026). This edition also describes Short-Term asking only the role and timeline (§3.2) on the dev branch, which reach the live site with the next deploy. The build a screen is running is shown in its ? menu, and [/api/v1/health](#) reports the same commit.

Part 1 — The engine

1.1 The cycle, and its nine phases

Everything in the performance module hangs off one row: the active cycle in `pms.cycles`. Its `phase` is the clock the whole tenant runs on.

`server/modules/performance/phase-machine.js` — pure, no database, so the rules are unit-tested directly.

```
draft → kra_open → mid_year_review → self_appraisal → manager_eval
      → hod_eval → calibration → publish → closed
```

- **Forward one step at a time.** `canAdvance` refuses any jump; HR cannot skip from `kra_open` to `calibration`.
- **Rollback one step**, HR-controlled and audited by the caller.
- **Cancel** from any phase except `closed`.
- `closed` is terminal — no advance, no rollback.

Stored values never changed when labels did. `self_appraisal` is shown as **Annual Review**; `kra_open` is shown as **KRA Setting and Growth Planning**. Renaming the stored value would rewrite every cycle row, audit entry and notification ever sent, to change a word that only `phaseLabel()` displays.

What each phase opens

Phase	Actions it permits
<code>kra_open</code>	<code>kra_edit</code> , <code>kra_submit</code> , <code>kra_decide</code>
<code>self_appraisal</code>	<code>self_edit</code> , <code>self_submit</code>
<code>manager_eval</code>	<code>manager_edit</code> , <code>manager_submit</code>
<code>hod_eval</code>	<code>hod_edit</code> , <code>hod_submit</code>
<code>calibration</code>	<code>calibrate</code> , <code>adjust</code> , <code>top_talent</code>
<code>publish</code>	<code>publish</code>

Two families sit outside that table on purpose.

KRA actions are open in every phase except `draft` and `closed`. The lock moved off the cycle and onto the sheet (§3.1). A draft cycle has not been opened to anybody; a closed one is history. Everything between is open, and the sheet's own status decides who may write.

Mid-year actions open at a phase and stay open (`OPEN_FROM`): from `mid_year_review` through to `self_appraisal`, the last phase an employee may still be writing in. They do not close when the phase passes.

Why employee windows stay open. The earlier rule was "editable only once the cycle is in this phase", which meant HR could not advance the cycle without shutting the door on anybody mid-way through. The

only remedy was a tenant-wide rollback — which reopens it for *everyone*, the opposite of locking. Now a window opens on a phase and closes on **your** act: you submitted, or your manager approved.

1.2 Who may open a screen

One row in `core.page_permission` drives **both** the menu and the direct-URL guard. They cannot disagree, because they are the same row.

```
tenant_id | page | route | required_permission
```

`NULL` = public, registered consciously. The eight "my own" pages are every employee's own record; Improvement Plan is row-scoped in its handler; My Surveys and People Hub are everyone's.

Effective permissions = `role_permissions[role]` U `user_permissions[email]`, with `*` as wildcard. Default bundles (`migrations/002`):

Role	Permissions
employee	<code>pms_self</code> , <code>engagement_take</code> , <code>people_view</code>
manager	+ <code>pms_team_eval</code>
hod	+ <code>pms_hod</code>
hrbp	<code>pms_self</code> , <code>pms_hrbp</code> , <code>people_view</code> , <code>engagement_take</code>
hr	<code>pms_admin</code> , <code>pms_team_eval</code> , <code>pms_hod</code> , <code>pms_hrbp</code> , <code>engagement_admin</code> , <code>people_admin</code> , <code>letters_admin</code> , <code>pms_compensation</code> , ...
admin	*

Bundles are **data, not code**. A client edits them; they never fork. They are re-seeded at every boot with `ON CONFLICT DO NOTHING`, so a missing grant is restored and an added one is kept. (Until 6 Oct the `hrbp` bundle was granted only to tenants that existed when migration 068 ran, so a fresh install's HRBPs held no HRBP access; it is now in the boot seed.)

Three layers, in order (`core/auth.js`, `core/permissions.js`):

1. **authenticate** — JWT, active employees only.
2. **apiPermissionParity** — a route table matched by method + longest path prefix. A 403 from here names what was needed in a `needs` field, which is what makes the next misconfiguration self-diagnosing.
3. **handler guards** — row-scoped decisions the table cannot express ("my team only", "HR-only section").

The carried lesson. Route rules cover coarse, all-or-nothing routes only. A flat prefix across mixed read/write routes denies users the handler would have allowed, so reads and writes get separate method-scoped rows. Row scoping lives in handlers, full stop.

1.3 Whose rows you see

Permission answers *may I open this screen*. Scope answers *which people are on it*, and they are different questions.

Who	Sees
Employee	Their own records
Manager	Their direct reports (<code>core.employees.manager_id</code>)
HOD	Their department, via <code>core.department_heads</code>
HRBP	Their remit — see below
HR / admin	Everybody in the tenant

The HRBP is the interesting one. An HRBP opens HR's own screens on parallel `/hrbp/*` routes, and sees only their own people on each.

`server/modules/performance/hrbp-gateway.js` is one gateway in front of the whole module rather than edits in 104 handlers:

- **LENT** — `pms_admin`, `people_admin`, `engagement_admin`, granted for the life of one request. Named, not a wildcard, so adding one is a decision somebody had to write down.
- **TENANT_WIDE** — settings, cycles, the KRA library, competency framework, the onboarding holidays, activity matrix and SPOC list, and similar. Configuration is the tenant's, not a person's: an HRBP may read it and may not change it.
- **Writes must name a person in the remit.** The gateway resolves every id in the request — an employee, a KRA sheet, a connect, an RnR nomination, an onboarding joiner or task — to the person it belongs to, and refuses the write if that person is outside the remit or cannot be resolved.
- **HR_ONLY** — `/hrbp/admin`. Reading is not automatically safe: lending `pms_admin` would otherwise open the screen that *sets* remits, letting an HRBP widen their own.

The remit itself lives in `core.hrbp_scope` (`kind` = location or HOD, `value`). Routes are separate from `/admin/*` on purpose: one page row drives both menu and guard, so giving an HRBP the `/admin/*` rows would put the **HR section itself** in their menu.

1.4 Four house rules visible on every screen

1. **Deterministic numbers; AI narrates.** Every rating, score and distribution is SQL. The agentic module drafts *text*, always labelled a draft, stored with the input that produced it. No AI output is ever parsed into a rating field.
2. **Thresholds, labels and dropdowns live in tables.** Rating scales, grade bands, increment ranges, competency levels, RnR awards — all editable rows. A client configures; they never fork.
3. **No silent failure.** Every batch operation returns per-row reasons. Boot fails on a migration error: a schema the code expects but does not have is a broken deploy, not a degraded state.
4. **Every change that affects a rating is audited.** `core.audit_log` plus explicit adjustment records, so “*why did my rating change?*” always has a queryable answer.

Part 2 — The five role sections

Since 6 Oct the screen is a white top bar (product name, page search, notifications, help, account) over a navy sidebar. The sidebar starts with **Dashboard** and then five role sections — **Self, Manager, HOD, HRBP, HR** — each opening into its pages. They are groups in `frontend/src/App.jsx`; an item appears only if the viewer holds its page permission, and a section whose every item is filtered out disappears entirely. The search box finds only pages the viewer may open. The help menu shows the build the screen is running, and a tab left open across a deploy shows a *Reload now* bar (`BuildWatch`).

2.1 Self — 13 pages plus Dashboard, everyone

Page	Route	What governs it
Dashboard	<code>/home</code>	Top of the sidebar, §2.6
My KRAs	<code>/my/kras</code>	Sheet status machine, §3.1. Weights must total exactly 100 to submit. Each KRA shows its timesheet hours and rating, §3.7
My Growth	<code>/my/growth</code>	Opens on your KRA submission, §3.2. Short-term and long-term aspiration are separate records; long-term stays editable through Manager Evaluation
Connects	<code>/team/connects</code>	1-on-1 log. Action items carry <code>sort_order</code> — <code>created_at</code> ties inside one transaction
Mid-Year Review	<code>/my/midyear</code>	Opens at <code>mid_year_review</code> , stays open to end of Annual Review. Per-KRA ratings with a computed overall
Annual Review	<code>/my/self-appraisal</code>	Opens at <code>self_appraisal</code> . Locks permanently on sign-off
Final Rating	<code>/my/annual-review</code>	Consolidated read-only view
My Rating	<code>/my/rating</code>	Gated: opens only once something is published for you. Not a permission — an emptiness
Past Cycles	<code>/my/history</code>	Published history plus imported prior-year ratings
My Competencies	<code>/my/competencies</code>	§3.8
Timesheet	<code>/my/timesheet</code>	Read-only to the employee. Your manager decides which KRA each item serves
Improvement Plan	<code>/pip</code>	Public page, row-scoped in the handler: your own plan, or your reports'
My Surveys	<code>/engagement</code>	§3.11
People Hub	<code>/people</code>	Company noticeboard

My Rating's gate is a "nothing here yet", not a permission. The page row is public, so typing the URL must *not* answer with the access-denied screen — that would tell somebody they had lost a right they never lost. It

says "No published ratings yet" and what will change that.

2.2 Manager — 9 pages, `pms_team_eval`

Page	Route	What governs it
Manager Dashboard	<code>/team/dashboard</code>	The Dashboard one scope out. First in the section, so opening Manager lands here
Nominate for RnR	<code>/rnr/nominate</code>	\$3.10
Team Overview	<code>/team/overview</code>	Every report, all phases at a glance
Team KRA Sheets	<code>/team/kra-sheets</code>	Each KRA shows the report's timesheet hours and rating, \$3.7. Approve or return. A return must carry a comment — refused 422, <i>"the employee must know why"</i> (<code>approvals.js</code>)
Team Target Achievements	<code>/team/growth</code>	Growth-plan decisions
Team Mid-Year	<code>/team/midyear</code>	Manager half of the checkpoint
Team Evaluation	<code>/team/eval</code>	Opens at <code>manager_eval</code> . Each KRA shows its timesheet rating beside the manager's buttons, \$3.7
Team Competencies	<code>/team/competencies</code>	Manager assessment per report
Timesheet	<code>/team/timesheet</code>	Where the mapping is done, \$3.7

Scope is the reporting line. When a reporting manager changes, open records move with the employee (`manager-handover.js`) — otherwise a submitted sheet sits in the queue of somebody who no longer manages them.

2.3 HOD — 2 pages, `pms_hod`

Page	Route	What governs it
HOD Review	<code>/hod</code>	Opens at <code>hod_eval</code> . Shows employee, manager and timesheet ratings per KRA
RnR Approvals	<code>/rnr/approvals/delivery-head</code>	Stage 2 of \$3.10

The `hod` role grants access to the screen; `core.department_heads` decides **which department's** evaluations appear. A person with the role and no mapping has a permanently empty queue — which is why the HOD screen exists and why a department can be given a head before anyone is in it.

2.4 HRBP — 20 pages, `pms_hrbp`

Every HR screen, on `/hrbp/*` routes, narrowed to the partner's own people by the gateway in \$1.3. All Approvals, RnR Approvals, Cycles, Employees, HOD, Career Pathing Matrix, KRA Overview, KRA Library,

Competency Framework, Competency Dashboard, Timesheet, PMS Completion Report, Calibration, 9-Box Grid, Closure Letters, Increment Simulation, Super 50, Engagement Surveys, New Hire Insights, Settings.

Read-only for the HRBP where the thing is the tenant's rather than a person's: Cycles, HOD, KRA Library, Competency Framework, Settings.

Increment Simulation is reachable because salary sits behind its own permission, `pms_compensation`, which the hrbp role is granted explicitly. That was a decision, not a tidy-up: "HR access" and "may see what people are paid" are different questions at most clients. Revoking it is one DELETE from `core.role_permissions`.

2.5 HR — 22 pages

All Approvals, Cycles, Employees, HOD, **HR Business Partners**, **RnR Final Approval**, **RnR Administration**, Career Pathing Matrix, KRA Overview, KRA Library, Competency Framework, Competency Dashboard, Timesheet, PMS Completion Report, Calibration, 9-Box Grid, Closure Letters, Increment Simulation, Super 50, Engagement Surveys, New Hire Insights, Settings.

Three are HR's alone: assigning HRBP remits, the final RnR gate, and RnR administration.

All Approvals (`approvals.js`) carries a distinction worth knowing:

- **decisions** — a sheet or plan somebody *submitted*, awaiting an approve or a return. Bulk-approvable, because approving is the whole action.
- **waiting** — a mid-year sign-off, a manager or HOD evaluation. Nobody has submitted anything; the named person still has to *write* their assessment. These carry no approve button, because bulk-approving one would mean writing an empty evaluation in someone else's name.

2.6 Dashboard

The Dashboard (`/home` , `HomePage.jsx`) answers *what now?*:

1. **Greeting band** — good morning/afternoon, the date, and the single most urgent thing, chosen server-side, with its link.
2. **Four attention cards** — what is outstanding, in the order it blocks people: evaluations to write, reports with no connect logged, employees with no manager, approvals pending, a returned KRA sheet, mid-year or annual review due, open connect actions. Only real counts get a card; with fewer than four, the row is completed with the person's own counts (KRAs and weight, connects, growth goals, rating) — never zeroes.
3. **Quick Actions** — four shortcuts on one line, chosen from pages the viewer may open, so none leads to an access-denied screen.
4. **Requested to manager** — what you are waiting on someone else for.
5. **My Team** (managers, and HR over everyone) — reports, KRA sheets approved, connects pending, evaluation completion; tabs for KRA approvals, evaluation status and connects; a department filter and name search; one row per person.

Removed on request: "My performance" (5 Oct — every link was already in the menu); and on 6 Oct the **Surveys waiting on you** list and the **PMS Cycle – Current Status** card ("*please remove these two tabs from homepage*"). Surveys are still on **Self** → **My Surveys** and in notifications. The cycle eligibility rule (§3.6) is still computed and returned by `/pms/home`; it simply has no card on Home now.

Part 3 — The machinery

3.1 The KRA sheet status machine

`pms.kra_sheets.status` :

Status	Who may write
<code>draft</code> / <code>returned</code>	The employee edits and submits
<code>submitted</code>	Locked to the employee; the manager decides
<code>approved</code>	Locked to everyone until HR reopens

Submission requires weights totalling exactly 100 (`weightsValid` , tolerance 0.01 for numeric drift).

The sheet is open for the whole cycle — every phase but `draft` and `closed` . The client's words: *"KRA should be open for all in entire cycle, but once KRA is submitted by employee it should be locked for him unless manager returns the KRA with any feedback."*

3.2 Growth planning opens per person

`growthEditable()` — the rule the cycle alone cannot express:

- In `kra_open` , it opens **when you submit your own KRA sheet**. Not when your manager approves it, and not when HR advances the tenant.
- After `kra_open` , the phase alone carries it, through Annual Review.
- A plan the manager **returned** is editable whatever the phase, up to Calibration — the return *is* the authorisation.
- `submitted` or `approved` is locked, and that outranks the window.

The intent of the older "don't write a plan against KRAs you are still inventing" rule survives in the submission condition: at that point the sheet is out of your hands, weights totalling 100 and all.

Aspiring Career, per horizon (`aspirationEditable()` , 8 Oct). The **Short-Term** tab follows the rule above. The **Long-Term** tab opens the same way and stays open **through Manager Evaluation**; from **HOD Review** on it is locked, whatever else has reopened — a returned plan included. Each tab has its own target role, plan and milestones (milestones typed on Long-Term used to be saved onto Short-Term). Milestone *progress* stays editable all year, on both.

Which roles a tab offers — the Career Pathing Matrix. Rows match on the employee's designation (From Role), department (blank = company-wide) and level. **Levels are compared as grade and band, not as text** (`people/career-level.js`): "Band 6", "E3" and "E3 · Band 6" are the same rung for an employee whose band is "E3 · Band 6"; "E2 · Band 6" or "Band 7" is not; blank means any level. Before 8 Oct the matrix's "Band 6" never matched the master's "E3 · Band 6", and employees were told their path "does not match your level". The same move written several times — once per department, or with differently written levels — is offered **once**, the employee's own department's row first. Short-Term offers one rung; **Long-Term walks up to three rungs** along rows HR wrote (each To Role and To Level becoming the next From), with times added up and

competencies combined. The target role is validated against the same list. HR's "matches N employees" counter uses the same level rule.

What each tab asks (8 Oct). **Short-Term** is the next move only, and it is **derived from the Career Pathing Matrix**: *Target role* must be a move configured from the employee's role and level — the server refuses anything else, and with no path configured there is nothing to choose (the screen says to ask HR; it used to accept any typed role) — and *Expected timeline* is the matrix's typical time for that move, stored by the server whatever is sent and shown read-only (typed only where HR left the figure blank). **Long-Term** asks the rest — total years of experience, skills and interests, growth plan and milestones. Saving Short-Term sends only its two fields, and a field a save does not send is kept, so anything an older version stored on Short-Term is not wiped (it is simply not shown). The Short-Term AI reads experience and skills from the Long-Term tab for its readiness read, and ignores any growth plan left on Short-Term.

Expected timeline comes from the matrix (8 Oct). Choosing a target role — from the list or with *Use this one* — sets Expected timeline to the matrix's typical time for that move (on Long-Term, the whole climb), shown underneath with the minimum ("typically 24 months for this move, at least 18 months"). A changed role no longer keeps the timeline typed for the previous one; a growth plan written for a different role is flagged ("This plan was written for ... — update it"). **A saved goal that is not on the tab's list** is shown as such rather than silently replaced. On Short-Term it is usually where the person wants to end up, saved before the matrix matched; **Move it to Long-Term** (`POST /people/career/my-path/move-to-long-term`) takes its role, timeline, plan and milestones there in one step and clears Short-Term for the next move (experience and skills stay on both). It is refused when Long-Term already has a goal. A long-term goal already saved may be kept while HR has not yet written the steps to it; a new long-term target must come from the matrix.

Where could I aim next? (AI). Sends the tab's horizon and **what is on the form, saved or not** — target role, timeline, growth plan, years of experience, skills and interests (a blank field falls back to what was last saved on that tab). The model starts from those, assesses a named target role first, and may only propose roles the matrix offers for that tab. When nothing matches, the box above the form says why — none configured, other departments only, level, or deactivated. **Use this one** sets the target role and fills **only blank fields** (timeline, growth plan); it never overwrites what the employee wrote, and adds suggested milestones not already listed, undated.

Long-Term builds on Short-Term (8 Oct). The Long-Term tab shows the saved short-term goal it continues from ("Builds on your short-term goal: Senior Software Developer in 12–18 months"), and its experience and skills start from the short-term answers when blank — the same person on both tabs. A Long-Term suggestion is sent the short-term target role, timeline, plan and milestones; each matrix rung that runs *through* the short-term target is marked, and the model proposes the role that comes **after** it — never the short-term role itself — with the time from now for the whole climb, first steps to start alongside the short-term plan, and milestones that follow the short-term ones without repeating them. **Use this one** fills the blank long-term fields: target role, timeline, growth plan, milestones, and experience and skills from Short-Term. With no short-term goal yet, the tab says to set it first.

3.3 Ratings: numbers in the database, letters on screen

Ratings stay **numeric** in storage — a weighted average of per-KRA scores is arithmetic. Letters are a presentation layer (`grade.js` server-side for PDFs and exports, `frontend/src/grade.jsx` for screens; two copies because they cannot import each other, and a letter on screen with a bare number in the PDF the employee keeps would be the worst of both).

The scale is a table on the cycle (`rating_scale`), not a constant.

A rating can move three times — manager, then HOD, then calibration — so every change is audited with its reason.

3.4 Calibration and the kitty

`calibration-budget.js`. **Money is integer paise throughout**, converted only at the edges: a thousand employees at 7.5% summed as floats drifts from the figure anyone gets adding the column in a spreadsheet, and a budget comparison off by rounding is worse than none.

Salary-bracket filter, a live kitty and budget panel, per-grade increment ranges, three pools, and a grid computing revised CTC per person. Two sanity ceilings: a percentage over 100 and a lump sum over ₹100 crore are treated as typos.

Grade bands are editable on the page — thresholds in tables, not code.

3.5 Increment simulation models pay, it does not set it

`increment-rules.js`. Nothing in it or its routes writes to `pms.compensation`. A simulation that could quietly become somebody's actual salary is a different and far more dangerous feature than the one asked for; the absent write path is what makes "run a few scenarios" safe.

Band ranges are inclusive at both ends — how people write them ("4 to 5 gets 10%") — and overlaps are rejected at save time.

3.6 Who is in this cycle

`eligibility.js`, from the client's rule: *joined on or before 31 Dec 2026 → July 2027 appraisal; joined on or after 1 Jan 2027 → July 2028.*

Implemented generally, not as two fixed dates, so July 2029 needs no code change. The cycle card states both halves, then tells **you** which applies — so somebody *not* in this cycle is told when theirs is, instead of working it out.

No date of joining on the master means eligibility cannot be computed, and the card says exactly that rather than guessing.

3.7 Timesheet → KRA

Three pure modules, deliberately separate: matching (`timesheet-kra-match`), scoring (`timesheet-kra-score`), rules (`timesheet-rules`). Attribution has to be arguable on its own, and a scoring change must never quietly move where somebody's hours were said to go.

What ships for the sheet as a whole is **coverage reporting**, plus a **rating per KRA** (below):

Number	Meaning
mapped %	of hours considered, how many are placed against any KRA
weighted coverage %	of the KRA sheet by weight , how much saw work
alignment %	how closely effort matched the weights

Number	Meaning
score / grade	withheld unless HR enables it <i>and</i> the mapping is thick enough to mean anything

Weighted coverage rather than counting buckets, because counting rewards breadth over value: on the real client month, every hour in one KRA scored 14%, while an hour in each of seven would have scored 100%.

The monthly rollup feeds Calibration as a **suggestion only**.

How an hour reaches a KRA, in order ([timesheet-kra-match.js](#)):

1. **An explicit mapping** — a manager or HR said "this item is that KRA".
2. **The KRA's own name** — the item is named after a KRA on the person's sheet (ignoring case, spacing and a trailing full stop; a title of 12+ characters also matches when it appears inside a longer item name). Added 6 Oct after a client upload renamed items to KRA titles and still showed 0%: until then only keywords matched.
3. **A keyword** — the KRA's keywords appear in the item name or description.
4. **Nothing** — the hours stay unplaced and the screen says so. Two KRAs matching the same item give it to neither ("ambiguous").

One Item Id, several names. When the same Item Id carries different item names in a period (the client's U5P-I58 was both *GFF Activities* and *Code review, security & architectural compliance*), each name is its own item, so a renamed row is not swallowed by the first name. A mapping saved on the bare Item Id still applies to every name under it, except where the name is a KRA title.

Timesheet rating per KRA (6 Oct)

Specified by the client: "if 1 KRA weighs 25%, it will be based on ratings like A+, A, B+, B as per no of hours worked ... I have to work 140 hours and I have worked 70 hours ... <40% will have B rating, <80% will be A and <100% will be A+." Ladder confirmed by the client the same day: "A+ more than 100%, A more than 80%, B+ more than 70%, B below 69%" — each band starts at its figure, so working exactly the expected hours is A+.

Step	Formula
required hours	working days in the period × hours per day (8 by default; weekends and the timesheet holiday list skipped)
KRA expected hours	required hours × the KRA's weight ÷ weight of the KRAs measured from timesheets
KRA effort %	hours placed against the KRA ÷ its expected hours (not capped — overtime shows, e.g. 125%)
rating	A+ 100% and above, A 80–99.9%, B+ 70–79.9%, B below 70% (editable as kra_bands)

Example: 140 h required, a 25% KRA expects 35 h. 17.5 h on it is 50% → B; 25 h is 71% → B+; 28 h is 80% → A; 35 h or more is 100%+ → A+. A KRA *not measured from timesheets* expects no hours and its weight is shared across the others. Hours not yet mapped to a KRA count for none, so ratings rise as items are mapped.

Where it shows: on **My KRAs** and **Team KRA Sheets**, under each KRA (hours worked, hours expected, %, rating, with a month picker and the formula above the table); on the **Timesheet** page; and beside each KRA on **Team Evaluation** and **HOD Review**.

Periods: the **Timesheet** page rates one month (to today at most). Team Evaluation and HOD Review rate the **days the uploads cover** within the cycle, so a month nobody uploaded is not counted as hours not worked.

It is **evidence, never the rating of record** — "shown beside, manager decides". Nothing writes it into an evaluation.

Who sees it: the employee (their own), their manager, the HOD of their department, and HR (`GET /api/v1/pms/timesheet/kra/ratings/:employeeId`).

Configuring it: HR → Timesheet → Settings → **KRA rating from the timesheet**. Hours per working day, the "indicative below this % mapped" threshold, and the bands (label and starting %; add or remove a band), with a worked check line that recomputes as you type. Saved to the timesheet scoring settings (`core.admin_settings` , key `timesheet` , `scoring`) through `PUT /api/v1/pms/timesheet/kra/scoring` , which validates them (every band needs a label, one band must start at 0, hours per day 0–24), audits the change, and leaves the overall score's own switch as it was. The working-day calendar — cycle start day and holidays — is the card above it. An HRBP cannot change either: they apply to everybody.

3.8 Competencies

A framework of competencies with levels per role, an employee self-assessment and a manager assessment. Technical skills are captured by tier — Primary, Secondary, Tertiary, Basic — rather than one proficiency per competency.

Competency mapping also supplies the potential axis the 9-box needs (`nine-box-derive.js`): before it, the grid showed only cells HR typed, because a derived placement needs performance bands and this repo keeps thresholds in tables rather than inventing cut-offs in a query.

3.9 Super 50

`super50.js` , from the client: *"ratings will be derived from last three annual reviews and ratings should be A or A+ with current year ratings as A+."*

Default rule `{ window: 3, minGrade: 'A', latestGrade: 'A+' }` — and it no longer hardcodes which **numbers** mean A and A+; it reads the tenant's scale. The window counts published annual history plus prior years imported from whatever the client appraised on before this product.

3.10 Rewards & Recognition

Three engines, separate on purpose: **eligibility**, **quota**, **workflow**.

Workflow — `draft → pending_delivery_head → pending_hrbp → pending_hr → final_approved` . Reject and send-back require a reason. A sent-back nomination can be resubmitted; a rejected one cannot.

Eligibility is a rule engine, not hardcoded categories: it is handed an employee, an award row and the tenant's settings. Every refusal is a full sentence naming the rule, the actual value, and when it changes — because a screen that says "not eligible" and stops sends the manager to email HR, which is the manual process with extra steps.

Experience windows are **half-open** `[min, max)` . The client spotted the overlap themselves: Rising Star is 1–3 years, Buddy Star 3+, so exactly 3.0 would qualify for both and the winner would be whoever nominated first. Half-open makes 3.0 Buddy Star, always.

Quota — 3% per cycle, one consolidated pool HR allocates across levels, consumed only at the final HR gate. Loyalty awards sit **outside** it: a 10-year award is a fact about a date, not something won in competition, and refusing the fifty-first person to reach ten years in a cycle of forty-two would mean declining to recognise someone for having worked there.

3.11 Engagement and anonymity

Anonymity is structural. Invitations and responses are separate tables and nothing may join them for an anonymous survey; the agentic themes feature reads a view with no identity columns.

Exports apply three rules on the way out, because once a file leaves the product nothing is enforced:

1. Unattributed answers never appear beside a name.
2. Aggregates of fewer than **5** responses are withheld.
3. An HRBP's export is scoped to their people and labelled as such.

External reviews (AmbitionBox, Glassdoor) arrive by **spreadsheet import**. There is no feed: Glassdoor retired its public review API in 2021, AmbitionBox never had one, and both prohibit scraping. Such surveys are `import_only` — they take responses but never invite anybody.

3.12 First-Week Journey — onboarding tracker (New Hire Insights, HR and HRBP; HR Ops)

The client's *7 Days Onboarding Tracker* workbook, as a tab on New Hire Insights (opens first; *Survey Insights* is the other tab). The HR Ops team has the same tracker as its own menu entry, **HR Ops → First-Week Journey** (`/hrops/onboarding`), hidden for anyone who can already open New Hire Insights.

Tables: `people.onboarding_activities` (the 48-row matrix, with the `sender_role` each task's email is sent from), `_days`, `_holidays`, `_joiners` (with `personal_email`), `_tasks`, `_feedback`, `_feedback_questions`, `_spocs`, `_task_emails` (with `from_email` and `sender_role`) — migrations 081, 082 and 083. The screen: a **Report date**, buttons for **Activity matrix**, **SPOCs**, **Holidays** and **Add joiner**, five cards, *By owner*, *By day of the journey*, and the joiner list; a joiner opens to their week, day by day, then Day-7 feedback.

- **A joiner is an employee** picked from the master (anyone active who joined in the last three weeks or joins in the next six is offered; anyone else by search). Manager, department and designation come from the master. Buddy and HR POC are chosen.
- **Starting a joiner creates one task per active activity** — 48 from the client's Activity Matrix.
- **Planned date** = `WORKDAY(DOJ, offset, holidays)`: weekends and the Holidays list are skipped; offset -2 is Pre-Day 1, 0 is Day 1, 6 is Day 7. Never stored — a holiday added later moves every plan.
- **Status** on the Report Date: *Completed* (has a completion date), *Overdue* (planned date passed), *Due Today*, *Upcoming*. Days overdue = `NETWORKDAYS(planned, report date) - 1`.
- **Joiner status**: *Completed* when every task is; else *N overdue*; else *On track*. Where: Not joined / Pre-Day 1 / Day n / After Day 7.
- **Day-7 feedback**: 1–5 on seven statements; average shown per joiner and across joiners.
- **Dashboard**: joiners in onboarding, due today, overdue, average feedback, completion %, by owner (a shared activity counts for every owner group named), by day.
- **Who may open it**: `engagement_admin` (HR) or `onboarding_ops` (`people/onboarding.js`, `guard`). **HRBP** sees and edits only joiners in their remit — the counts too. Holidays, the SPOC list and the matrix are

company-wide, so an HRBP reads them and HR changes them.

- **The tick belongs to HR Ops, HR and HRBP** (corrected 7 Oct). Marking a task done or not done, its completion date, *Mark due ones done*, the acknowledgement and sending a joiner email need `onboarding_ops`, which the `hr_ops`, `hr` and `hrbp` bundles carry (migrations 002 and 083; `opsGuard`). Anyone else who can open the tracker sees the ticks greyed, with "HR Ops, HR and HRBP mark tasks done", and can still add remarks, issues and closure dates. The server refuses the fields regardless of what the screen shows (403, `needs: onboarding_ops`). `hr_ops` is a role HR assigns on **Employees** like any other.
- **Every task's email goes TO the joiner, FROM the SPOC who owns the activity** (corrected 7 Oct; it used to go to the SPOC). *Email joiner · from {role}* on each task opens the draft — "Dear {first name}", what happens, when, what it is for, signed by the SPOC — editable before it goes. Every send is kept on the task with who it was from and to.
- **Which SPOC** — the activity's `sender_role`, the first owner named on the client's matrix (migration 083, `SENDER`): e.g. the intimation mail from the **Recruiter**; HR Welcome, Documentation and the HRMS/policies walkthrough from **HR Ops**; Workplace Tour from **Admin**; IT Setup and Access Validation from **IT**; Team Introduction and the KPI/KRA discussion from the **Manager**; Buddy Introduction from the **Buddy**; Code of Conduct and the 7-day connect from **HR**; Business Overview and PMS Orientation from the **HRBP**; Learning Needs from **L&D**. Where the matrix names two owners (*Manager/SME, HR Ops → IT*) the first sends it. The matrix shows it in an *Email from* column. The address: Manager and Buddy from the joiner's record; HR from the joiner's HR POC, else the HR desk; every other role from the **SPOCs** list on this page. A SPOC with no address set is named on screen and nothing is sent.
- **Which address of the joiner's** — before the date of joining, the **personal email** HR records on the joiner's week (the company mailbox usually does not exist yet); from day one, the company address from the employee master; the personal one if the master has none. With neither, the screen says so and nothing is sent. Both addresses are worked out on the server, never taken from the request.
- **How it leaves: the SPOC sends it from their own Gmail — no setup** (decided 7 Oct: "avoid all these setup and share mails directly from spocs mail"). Every employee's **Home** page carries *Onboarding emails for you to send* when they are the SPOC for any (`GET /people/onboarding/my-emails`): the activities whose sender works out to *their* address — the activity's SPOC role from the SPOC list, or the joiner's own manager, buddy or HR POC — not yet done, not yet sent, due within seven days or overdue; five shown, most urgent first, with *Show all*. **Open in Gmail** opens Gmail's compose window in their own browser and Google account (`authuser`), addressed to the joiner with the draft filled in; they send it there and press **I've sent it** (`POST /tasks/:id/mark-sent`). The PMS cannot see anyone's Gmail, so "sent" is the SPOC's word, logged as `own_gmail` / `sent_by_spoc` with who said so and when, and audited. HR Ops, HR and HRBP (in remit) can **Mark as sent** on the SPOC's word from the tracker, where each task's email box says whether it is waiting on the SPOC or was sent and when. Nobody else can mark another person's email sent.
- **Optionally, the PMS sends them itself** once HR has set email up under Settings → Email and switched it to Live (below): *Send through the PMS now* appears for HR Ops, HR and HRBP, still from the SPOC's own address (Google's SMTP relay, a Google key, or a mailbox with Send As).
- **Email: HR → Settings → Email — three steps**. Until email is *Live* (the default is *Simulated*), every email is logged in `core.notif_log` and none is delivered. 1. **Connect**. Three choices:
 - **Google Workspace** (Mindgate; the default when nothing else is set). **The simple way — Google's SMTP relay** (7 Oct, "make this more simple for IT"): IT, once, in the Google Admin console → Apps → Google Workspace → Gmail → Routing → *SMTP relay service*: allowed senders *Only addresses in my domains*; authentication *Only accept mail from the specified IP addresses* with this server's public IP (shown on the card, with a copy button — found from AWS's `checkip` service, or `SERVER_PUBLIC_IP`); *Require TLS*. No key, no password: the PMS connects to `smtp-relay.gmail.com:587` without signing in,

greet with the company domain (Google turns away an unqualified name), and sends from the person's own address — any address in the domain, group addresses included. HR sets the address reminders are sent from. The IP must stay fixed (an Elastic IP on AWS). Copies do not land in the SPOC's Sent folder. **Alternative — a service-account key** (a link on the card), for copies in each SPOC's Sent folder: IT enables the Gmail API in a Google Cloud project, creates a service account and downloads its JSON key, uploads it here, then in the Admin console → Security → API controls → *Domain-wide delegation* adds the **Client ID** (shown after upload) with the one scope

<https://www.googleapis.com/auth/gmail.send>. The PMS signs an RS256 token request for each person (`sub`) and posts to the Gmail API (`core/gmail.js`). That lets it *send* (never read) as any user in the domain — not groups — so it only ever sends as an address it worked out itself (the activity's SPOC, the joiner's manager, buddy or HR POC, or the reminders sender), never one from a request, and logs every send. The key is write-only and never audited.

- **Microsoft 365 / Other**: one mailbox and its password; the provider fills in server, port and STARTTLS (*Advanced (for IT)* holds the rest). It sends onboarding emails as the SPOC, needing Send As from IT. `SMTP_*` / `MAIL_FROM` in `api.env` still work. 2. **Send a test email to yourself** — really delivered even in Simulated mode, only to the person pressing it, through the same transport every email uses; gives up after 20 seconds; a failure comes back as a sentence (`core/mail.js` `explain` : Google not yet authorised, Gmail API off, not a Workspace user, Authenticated SMTP off, wrong password, server unreachable, Send As refused) with the server's own words beneath. With Google, **each SPOC address is checked as well, without sending anything** (`GET /people/onboarding/spocs/check`): Google refuses a token for a group, a typo or an outsider. 3. **Go live** — offered only after a delivered test; the server refuses it otherwise. Saving new connection details, or switching between Google and a mailbox, throws the old test away (and a switch while live drops back to recorded-only until tested).

The badge says where HR is: *Not set up, Next: send a test email, Test failed, Ready to go live, Live*. The card is HR's alone — an HRBP does not see it.

Verified against all 288 real rows of the workbook: planned date, status and days overdue match on every one. Three workbook faults were corrected rather than copied: the Day column (Readiness rows said Day 1 but were planned before joining), 55 tracker rows per joiner for 48 activities, and the Ownership sheet counting "Recruiter" as IT.

3.13 Reminders

`reminder-schedule.js` is the pure calendar; `reminders.js` answers who it is about and whether they already got it.

1. quarterly connect — employee + manager, first Monday after each quarter
2. mid-year self — 1 September, then 15th/20th/25th/last Friday
3. mid-year manager — first Monday of September, then the same
4. mid-year chase — every weekday from 3 days after the employee signs until the manager finalises
5. annual — the same three shapes again, in March

Plus a self-chase after **7 days** on a form nobody has filled in, measured from the phase opening (the latest `PHASE_ADVANCE` in `pms.audit_log`).

Part 4 — Changing behaviour without code

What	Where
Who may open a page	<code>core.page_permission</code>
What a role may do	<code>core.role_permissions</code> , <code>core.user_permissions</code>
Rating scale, bell curve	<code>pms.cycles.rating_scale</code> , <code>bell_curve</code>
Grade bands, increment ranges	Calibration screen → <code>pms.grade_ladder</code> , <code>pms.increment_matrix</code>
Competency levels	<code>pms.competency_scale</code> , <code>pms.competency_role_levels</code>
RnR awards, bands, quota	<code>rn timer.awards</code> , <code>rn timer.band_levels</code> , <code>rn timer.settings</code>
Connect questions	<code>pms.connect_questions</code>
HRBP remits	<code>core.hrbp_scope</code>
Department heads	<code>core.department_heads</code> (HR → HOD)
Super 50 rule, KRA Library scope	HR → Settings
Timesheet calendar (cycle start day, holidays, compliance thresholds)	HR → Timesheet → Settings
Timesheet KRA rating bands, hours per day, indicative threshold	HR → Timesheet → Settings → KRA rating from the timesheet
Which KRA a work item serves; KRAs not measured from timesheets	Manager → Timesheet (mapping), HR mapping backlog
Onboarding activity matrix, day themes, Day-7 statements	<code>people.onboarding_activities</code> , <code>_days</code> , <code>_feedback_questions</code> (seeded from the client's workbook)
Onboarding holidays, SPOC email addresses (who each email is sent from)	New Hire Insights → First-Week Journey → Holidays / SPOCs
Which SPOC sends each onboarding activity's email	<code>people.onboarding_activities.sender_role</code> (seeded from the client's matrix)
A joiner's personal email (used before joining)	The joiner's week → <i>Personal email (before joining)</i>
Who ticks onboarding tasks	Permission <code>onboarding_ops</code> — in the <code>hr_ops</code> , <code>hr</code> and <code>hrbp</code> bundles; role <code>hr_ops</code> set on Employees
Email for reminders and notifications (and, optionally, onboarding emails sent by the PMS itself): Google SMTP relay, Google key, or a mailbox; test; Go live. Onboarding emails need none of it — SPOCs send them from Home	HR → Settings → Email (three steps)

What	Where
AI on/off	<code>ANTHROPIC_API_KEY</code> in <code>/etc/agentics-pms/api.env</code> — instance-owned, never written by a deploy
AI model	<code>deploy/service/managed-settings.env</code> , pushed into <code>api.env</code> by every deploy (<code>UNMANAGED=AI_MODEL</code> pins a box)

Part 5 — What it deliberately does not do

- **It is not an HRMS.** The employee master is a mirror, loaded by validated import. Nothing edits it as a system of record.
- **It does not set pay.** Increments are modelled, never written.
- **AI never produces a number.** Ratings, scores and distributions are SQL; AI drafts text, labelled a draft, stored with its input.
- **No dummy data.** Empty states are honest.
- **A missing key is not a broken product.** With no `ANTHROPIC_API_KEY`, agentic endpoints answer a clean 503 and everything else works.
- **The timesheet does not set a rating.** Its per-KRA rating is shown beside the manager's and HOD's own; the people rate.
- **It does not pretend to send email.** In simulated mode an email is recorded and the screen says it was not delivered.

Checked against the code on 8 October 2026 (live: `4e9d532` ; plus the Short-Term asking only the role and timeline, on the dev branch). Where this book and the code disagree, the code is right and this book is a bug.